CAHIER DES CHARGES
Smart Web Extractor

Extension Chrome d'extraction de données avec apprentissage du parcours manuel

Élément	Description
Version du document	2.0 (révision corrigée de la v1)
Date	22/10/2026
Plateforme	Google Chrome ≥ 116 — Manifest V3
Technologies	TypeScript strict, React, Vite
Modes d'exécution	Semi-automatique et automatique (choisis au lancement)
Sources	Tableaux HTML, listes non tabulaires (cartes, grilles), fiches détaillées
Exports	Excel XLSX et CSV
Stockage	Local — `chrome.storage.local` (scénarios, préférences) et IndexedDB de l'extension (exécutions, résultats, journaux)

0. Historique des modifications (v1 → v2)

#	Sujet	Correction apportée
1	Modèle de données incomplet	Modèle entièrement refait (§5) : étapes structurées, localisateurs multi-stratégies, domaine, politique de reprise, limites.
2	Notion de campagne absente	Ajout de l'objet **Exécution** (`Run`) ; chaque résultat référence `runId` et la version du scénario.
3	Mode stocké dans le scénario	Le mode devient un paramètre de l'exécution. Le scénario déclare seulement les modes compatibles.
4	Popup inadaptée au pilotage	Pilotage déplacé dans un **panneau latéral** (`chrome.sidePanel`). La popup ne sert plus qu'au lancement rapide.
5	Contraintes MV3 non prises en compte	Nouvelle section §4.2 : service worker éphémère, IndexedDB, injection des content scripts, build.
6	Doublons, clés et associations ambigus	Règles explicites (§3.4, §3.3) : clé principale, clé de repli, signalement sans suppression, association manuelle.
7	Contradiction CSV : zéros initiaux / formules	Règles d'export tranchées (§3.7).
8	Erreurs non couvertes	Tableau d'erreurs complété (§7) : session expirée, CAPTCHA, onglet fermé, HTTP 403/429, boîtes de dialogue.
9	Périmètre de détection limité aux `<table>`	Ajout des listes non tabulaires, des fusions de cellules, des iframes, du Shadow DOM et des contenus virtualisés (§3.1).
10	Données personnelles	Nouvelle section §6.2 : conformité, durée de conservation, purge.
11	Critères d'acceptation non mesurables	Critères chiffrés et vérifiables (§8.3).
12	Termes non définis	Glossaire ajouté (§1.3).
13	Arborescence illisible dans le PDF	Arborescence reconstruite (§4.3).
14	Prompt Claude Code incomplet	Prompt mis à jour pour couvrir toutes les exigences (§10).

1. Présentation du projet

Smart Web Extractor est une extension Chrome conçue pour extraire des données depuis des sites web **auxquels l'utilisateur a légitimement accès**, sans écrire de script spécifique pour chaque site. L'utilisateur sélectionne les champs, enregistre un parcours de navigation, valide le scénario, puis lance une exécution dans le mode de son choix.

Principe fondamental : l'extension mémorise des **règles** d'identification des champs et des **étapes** de navigation, et non les valeurs observées sur la page d'apprentissage.

1.1 Objectifs

- Analyser une page et détecter ses tableaux, listes, colonnes, champs, libellés et liens utiles.
- Laisser l'utilisateur choisir les champs à extraire et prévisualiser les valeurs.
- Enregistrer explicitement un parcours manuel et le transformer en scénario structuré et modifiable.

- Exécuter une extraction en mode semi-automatique ou automatique.
- Associer les données de listes et de fiches détaillées au moyen d'une clé stable.
- Consulter, filtrer, corriger et exporter les résultats en XLSX ou CSV.
- Conserver localement les scénarios et les résultats, gérer les erreurs et permettre la reprise.

1.2 Scénarios d'utilisation

- **Liste seule** : sélectionner les colonnes, extraire les lignes et parcourir la pagination compatible.
- **Fiche seule** : détecter les champs d'une fiche et en extraire les valeurs.
- **Liste + fiches** : collecter les données de la liste, ouvrir chaque fiche, en extraire les champs et fusionner les deux sources à l'aide d'une clé stable.

1.3 Glossaire

Terme	Définition
Scénario	Ensemble versionné de règles (champs, localisateurs, étapes, pagination, association, limites). Il ne contient aucune donnée extraite.
Exécution (Run)	Une campagne d'extraction lancée à partir d'une version précise d'un scénario, dans un mode donné. Les résultats sont rattachés à une exécution.
Localisateur	Règle permettant de retrouver un élément du DOM, composée de plusieurs stratégies classées par fiabilité.
Étape	Action élémentaire d'un parcours (ouvrir une fiche, extraire, revenir, page suivante…) avec sa condition de réussite, son délai et sa politique de reprise.
Condition de réussite	Vérification **déclarative** exécutée après une étape (élément visible, URL correspondante, nombre de lignes modifié…). Jamais une expression de code.
Vérification	Contrôle automatique d'une condition de réussite par le moteur. Ce n'est pas une validation humaine, sauf en mode assisté.
Mode assisté	Repli du mode automatique : l'extension met l'exécution en pause et demande à l'utilisateur de réaliser l'étape lui-même, puis reprend.
Pagination assistée	En mode semi-automatique, l'extension détecte et met en évidence le bouton « page suivante », mais c'est l'utilisateur qui clique. L'extraction de la nouvelle page est proposée automatiquement.
Clé d'association	Champ (ou combinaison de champs) qui identifie un enregistrement de façon unique, utilisé pour fusionner liste et fiche et pour dédupliquer.
Doublon	Deux enregistrements d'une même exécution ayant la même clé d'association.
Valeur sensible	Valeur d'un champ `input[type=password]`, d'un champ dont l'attribut `autocomplete` désigne un secret ou un moyen de paiement (`current-password`, `new-password`, `one-time-code`, `cc-*`), d'un champ caché (`type=hidden`), ou d'un champ que l'utilisateur a marqué comme sensible.

2. Modes de fonctionnement

Le mode est **choisi au lancement de l'exécution** et n'est pas une propriété figée du scénario. Le scénario indique seulement les modes avec lesquels il est compatible.

Fonction	Semi-automatique	Automatique
Détection des champs	Oui, à chaque page	Relocalisation via les règles du scénario validé
Sélection / configuration	Par l'utilisateur	À partir du scénario validé (non modifiable pendant l'exécution)
Navigation entre les fiches	L'utilisateur	L'extension, si le scénario le permet
Extraction	À la demande, ou automatiquement à l'arrivée sur une page correspondant à une étape configurée	Selon les étapes du scénario
Pagination	Assistée (cf. glossaire)	Automatique si compatible
Vérification après action	Affichage du résultat à l'utilisateur	Contrôle automatique de la condition de réussite avant l'étape suivante
Erreurs	Signalées immédiatement à l'utilisateur	Journalisées ; pause automatique selon la politique de reprise
Export	À la demande	À la fin de l'exécution (proposé, jamais forcé) ou à la demande

Le mode semi-automatique doit rester utilisable lorsqu'une navigation complexe ou une limitation du site empêche l'automatisation complète. En mode automatique, si une condition de réussite n'est pas satisfaite après le nombre de tentatives prévu, l'exécution passe en **mode assisté**. Elle ne poursuit jamais à l'aveugle.

3. Spécifications fonctionnelles

3.1 Module A — Détection des champs

Sources détectées

- Tableaux HTML `<table>`, avec ou sans `<thead>` / `<th>`. Sans en-tête explicite, la première ligne est proposée comme en-tête et l'utilisateur confirme.
- Cellules fusionnées (`rowspan`, `colspan`) : les valeurs fusionnées sont répliquées sur les lignes et colonnes concernées.
- En-têtes sur plusieurs niveaux : les libellés sont concaténés (« Identité › Nom »).
- Tableaux imbriqués : seul le tableau de plus haut niveau sélectionné est extrait ; les tableaux internes sont signalés.
- Listes non tabulaires : éléments répétés de même structure (cartes, `<li>`, grilles de `div`, `role="grid"`, `role="row"`). Elles sont détectées par similarité de structure entre éléments frères.
- Champs `input`, `select` et `textarea`, avec leur libellé (`<label for>`, `aria-label`, `aria-labelledby`, texte voisin). **Les valeurs sensibles ne sont jamais lues** (cf. glossaire).
- Paires libellé–valeur : `<dl>/<dt>/<dd>`, lignes « Libellé : valeur », tableaux à deux colonnes.
- Liens, boutons et éléments cliquables utiles à l'ouverture des fiches ou à la pagination.

Contenus particuliers

- Contenus chargés dynamiquement : observation du DOM (`MutationObserver`) jusqu'à stabilisation, avec un délai maximal.
- Défilement infini et listes virtualisées (seules les lignes visibles existent dans le DOM) : détectés et signalés. L'extraction se fait par défilement progressif, avec déduplication par clé et une limite d'itérations.

- Shadow DOM ouvert : parcouru. Shadow DOM fermé : signalé comme non accessible.
- Iframes de même origine : prises en charge. Iframes d'une autre origine : prises en charge uniquement si l'utilisateur a accordé l'accès à ce domaine ; sinon elles sont signalées.

Sélection et aperçu

- La sélection se fait par un **sélecteur visuel dans la page** : surbrillance au survol et clic pour choisir. Cette surcouche est isolée dans un Shadow DOM afin de ne pas perturber la page.
- Chaque champ détecté affiche un aperçu de ses valeurs (au moins trois exemples s'ils existent). L'utilisateur peut le sélectionner, le renommer, le marquer comme obligatoire, le marquer comme sensible et choisir son type (texte, nombre, date, lien).

Localisateurs

- Chaque champ reçoit un localisateur réutilisable composé de plusieurs stratégies, essayées dans cet ordre de fiabilité :
- ID stable (les ID qui semblent générés, par exemple avec de longues suites de chiffres ou des empreintes, sont écartés) ;
- attributs stables (`data-*`, `name`, `aria-*`) ;
- rôle accessible et nom accessible ;
- libellé associé ;
- sélecteur CSS structurel ;
- contexte DOM (position relative à un en-tête ou à un libellé).
- Pour un champ de ligne (dans une liste), le localisateur est **relatif à la ligne** (par exemple « colonne dont l'en-tête est "Matricule" ») et non absolu.
- Quand une stratégie trouve plusieurs éléments ou que les stratégies ne s'accordent pas, l'extension **signale l'ambiguïté** et demande à l'utilisateur de choisir. Elle ne choisit jamais arbitrairement.

3.2 Module B — Enregistrement du parcours

- L'enregistrement est démarré et arrêté **explicitement** depuis le panneau latéral. Un indicateur visible (badge de l'icône et bandeau dans la page) signale qu'il est actif.
- L'enregistrement est limité à l'onglet et au domaine du scénario. Il s'arrête automatiquement si l'utilisateur change de domaine, ferme l'onglet ou reste inactif plus de 15 minutes (durée configurable).
- Types d'étapes enregistrés : `locateList`, `openDetail`, `extractFields`, `returnToList`, `nextPage`, `waitFor`, `scrollLoad`. Les autres actions (survols, clics sans effet, saisies) sont ignorées.
- **Saisie de texte** : jamais enregistrée par défaut. L'utilisateur peut déclarer explicitement une étape de filtre ou de recherche avec une valeur fixe non sensible. Les valeurs sensibles sont refusées.
- Pour chaque étape, l'extension enregistre : son type, son localisateur, sa condition de réussite, son délai maximal, sa politique de reprise et son mode d'ouverture (même onglet ou nouvel onglet, pour `openDetail`).
- Les actions sont converties en scénario structuré, consultable, modifiable et **testable sur la page courante sans enregistrer de résultats** (mode « essai »).
- Les règles s'appuient sur le DOM. Les coordonnées de souris ne sont jamais utilisées.
- Ne sont jamais enregistrés : mots de passe, cookies, jetons, en-têtes d'authentification, contenu de `localStorage` ou `sessionStorage` des sites.

3.3 Module C — Moteur d'extraction

- Extraire toutes les lignes d'une liste et les champs sélectionnés d'une fiche.
- Ouvrir les fiches liées à une liste, revenir à la liste et gérer la pagination compatible.
- **Stratégie d'ouverture des fiches** :
- si le lien a une URL (`href`), la fiche est ouverte de préférence dans un onglet en arrière-plan, puis l'onglet est fermé après extraction ; la liste ne perd ainsi ni sa position ni sa page ;

- sinon (clic JavaScript, modale, SPA), la fiche est ouverte dans le même onglet ; le retour se fait par `history.back()` ou par le bouton de retour configuré. Le moteur vérifie ensuite qu'il est revenu **sur la bonne page de la liste** et, sinon, il y retourne.
- Attendre les éléments attendus, y compris sur les pages dynamiques, jusqu'au délai maximal de l'étape.
- Détecter les doublons, les champs manquants et les enregistrements incomplets. **Les doublons sont signalés et non supprimés.** L'option `duplicates` du scénario indique s'ils sont exclus de l'export par défaut.
- Conserver pour chaque enregistrement : l'URL source, la page de liste d'origine, l'état de traitement et les erreurs.
- Séparer strictement les règles de navigation (étapes) des règles d'extraction (champs).
- Respecter un délai minimal entre deux actions et une limite d'actions par minute.

3.4 Module D — Association des données

- Fusionner les données de la liste et de la fiche en un enregistrement unique.
- **Choix de la clé, par ordre de priorité** :
- identifiant technique stable trouvé dans la page (attribut `data-id`, paramètre d'URL de la fiche, etc.) ;
- champ métier unique déclaré par l'utilisateur (matricule, référence…) ;
- combinaison de champs déclarée par l'utilisateur ;
- clé de repli : URL canonique de la fiche (sans paramètres de suivi).
- Si seule la clé de repli est disponible, l'extension affiche un **avertissement de risque d'association** avant le lancement.
- **Conflit de valeurs** : quand un champ est présent dans la liste et dans la fiche avec des valeurs différentes, la règle `conflictPolicy` du champ s'applique (`preferDetail` par défaut, `preferList`, ou `flag`). Le conflit est toujours tracé dans l'enregistrement.
- Ne jamais fusionner automatiquement des enregistrements sur la seule ressemblance des noms ou d'autres valeurs approximatives.
- Les associations incertaines sont marquées et peuvent être **confirmées, corrigées ou dissociées manuellement** dans le tableau de résultats.

3.5 Module E — Gestion des scénarios

- Créer, modifier, dupliquer, tester (mode essai), importer, exporter et supprimer des scénarios.
- Un scénario enregistre : le domaine autorisé, l'URL de départ, les champs, les étapes, la pagination, la clé d'association, les modes compatibles et les limites.
- Toute modification d'un scénario validé crée une **nouvelle révision**. Les exécutions passées restent liées à la révision utilisée.
- Un scénario peut être réutilisé autant de fois que nécessaire : chaque lancement crée une nouvelle exécution. Les résultats de deux exécutions ne sont jamais mélangés.
- **Import** : le fichier JSON importé est traité comme non fiable. Il est validé par un schéma strict (types, tailles maximales, aucune propriété inconnue). Le domaine qu'il contient doit être confirmé par l'utilisateur, et la permission d'accès est demandée si nécessaire.
- **Export** : il contient uniquement les règles, jamais de résultats ni de secret.
- Les scénarios ne contiennent jamais de secret d'authentification.
- Le champ `schemaVersion` permet la migration automatique des scénarios enregistrés avec une version antérieure du format.

3.6 Module F — Tableau de résultats

- Afficher les résultats d'une exécution, avec recherche, filtres par colonne et par état, et tri.
- Autoriser la correction manuelle d'une valeur : **la valeur d'origine est conservée**, la valeur corrigée est stockée à part et la cellule est marquée comme corrigée. On peut annuler une correction.
- Autoriser la suppression d'une ligne (suppression logique, réversible jusqu'à la purge de l'exécution).

- Signaler visuellement les doublons, les champs obligatoires manquants, les conflits de valeurs, les associations incertaines et les erreurs.
- Afficher l'**URL source** de chaque enregistrement (lien cliquable ouvrant la page dans un nouvel onglet).
- Exporter toutes les lignes, les lignes filtrées ou une sélection.
- Relancer une tentative sur les enregistrements en erreur, sans créer de doublon.
- Comparer deux exécutions d'un même scénario (évolution prévue en V2, hors périmètre V1).

3.7 Module G — Export

Règles communes

- Les données stockées en interne ne sont **jamais** modifiées par l'export. Les éventuelles transformations ne s'appliquent qu'au fichier produit.
- Pour chaque cellule, la valeur exportée est la valeur corrigée si elle existe, sinon la valeur d'origine. Une option permet d'exporter les deux valeurs.
- Colonnes techniques facultatives : clé, URL source, date de collecte, état, erreurs.

XLSX (ExcelJS)

- En-têtes clairs (libellés des champs), filtre automatique, première ligne figée, largeur de colonnes adaptée au contenu (bornée).
- Toutes les valeurs sont écrites en type **texte**, sauf pour les champs déclarés en type nombre ou date. Les identifiants à zéros initiaux sont ainsi préservés, et aucune valeur n'est interprétée comme une formule, sans avoir besoin de préfixe.
- Caractères français et arabes : encodage natif de XLSX. Une option « feuille de droite à gauche » est proposée et activée automatiquement si la majorité des valeurs est en arabe.

CSV

- Encodage UTF-8 **avec BOM**, indispensable pour qu'Excel affiche correctement les accents et l'arabe.
- Séparateur au choix : `;` (par défaut, adapté à un Excel en français), `,` ou tabulation. Guillemets conformes à la RFC 4180.
- **Protection contre l'injection de formules** (activée par défaut, désactivable) : toute valeur texte commençant par `=`, `+`, `-`, `@`, une tabulation ou un retour chariot reçoit une apostrophe `'` en tête. Les valeurs strictement numériques (par exemple `-12,5`) ne sont pas préfixées. Cette transformation est indiquée dans la fenêtre d'export.
- **Zéros initiaux** : le fichier CSV contient les valeurs exactes (`00123`). Excel peut supprimer ces zéros à l'ouverture par double-clic. Cette limite d'Excel est signalée à l'utilisateur, qui se voit recommander l'export XLSX. L'astuce `="00123"` est **interdite**, car c'est une formule.

4. Architecture technique

Composant	Technologie	Responsabilité
Extension	Chrome Manifest V3	Intégration au navigateur et permissions
Langage	TypeScript strict	Typage et maintenance
Build	Vite + plugin d'extension (`@crxjs/vite-plugin` ou équivalent, à valider en phase 1)	Compilation, génération du manifeste, bundles des content scripts
Interface	React	Panneau latéral (pilotage), popup (lancement rapide), page tableau de bord (scénarios, résultats)

Composant	Technologie	Responsabilité
Surcouche de sélection	Content script + Shadow DOM	Sélecteur visuel dans la page
Extraction	Content scripts injectés à la demande, API DOM	Détection et lecture des pages autorisées
Orchestration	Service worker	Coordination, messages, machine à états des exécutions
Paramètres, scénarios	`chrome.storage.local`	Scénarios et préférences
Exécutions, résultats, journaux	IndexedDB **de l'extension**	Stockage local de volumes importants
Validation	Bibliothèque de schéma (par exemple Zod)	Validation des messages et des imports
Excel	ExcelJS	Création des fichiers XLSX
Tests	Vitest (unitaires) ; Playwright avec Chromium en contexte persistant et extension chargée (bout en bout)	Tests unitaires et de bout en bout

Aucun backend n'est requis pour la première version. L'application fonctionne entièrement en local et n'envoie aucune donnée extraite à un serveur externe.

4.1 Interfaces utilisateur

Interface	Rôle
Panneau latéral (`chrome.sidePanel`)	Interface principale : analyse de la page, panneau de détection, enregistrement du parcours, lancement, pause, reprise, arrêt, progression en direct, demandes du mode assisté. Il reste ouvert pendant que l'utilisateur navigue.
Popup	Lancement rapide : ouvrir le panneau latéral et accorder l'accès au site courant.
Tableau de bord (page de l'extension)	Gestion des scénarios (éditeur), historique des exécutions, tableau de résultats, exports, journaux, paramètres et purge des données.

La popup se ferme dès qu'elle perd le focus : elle ne doit donc jamais être la seule interface de contrôle d'une exécution.

4.2 Contraintes Manifest V3 à respecter

1. **Service worker éphémère** : Chrome peut l'arrêter après environ 30 secondes d'inactivité. Il ne garde aucun état en mémoire seulement. L'état de chaque exécution (étape courante, page, index de la fiche, compteurs) est persisté **après chaque étape** dans IndexedDB, et le service worker sait reprendre à partir de cet état à son redémarrage.
2. **Content scripts détruits à chaque navigation** : le content script est sans état. Il reçoit ses instructions du service worker à chaque chargement de page.
3. **IndexedDB** : un content script qui ouvre IndexedDB accède à la base **du site visité**, pas à celle de l'extension. Les content scripts n'écrivent donc jamais directement : ils envoient les données au service worker par message.
4. **Injection** : les content scripts sont injectés à la demande avec `chrome.scripting.executeScript`, sans déclaration `content_scripts` à portée générale. Les scripts injectés ne peuvent pas être des modules ES : ils sont compilés en bundles autonomes (IIFE).
5. **Messages** : chaque message entre composants a un type déclaré et est validé par un schéma. L'expéditeur est contrôlé (`sender.id` égal à l'extension, onglet attendu). Les messages invalides sont rejetés et journalisés.

6. **Politique de sécurité du contenu (CSP)** : la CSP par défaut de MV3 est conservée. Pas de `eval`, pas de `new Function`, pas de script distant.

4.3 Permissions

Permission	Justification
`storage`	Scénarios, préférences
`scripting`	Injection des content scripts à la demande
`activeTab`	Analyse ponctuelle de l'onglet courant, sans permission permanente
`sidePanel`	Panneau latéral
`optional_host_permissions` : `https://*/*`, `http://*/*`	Demandées **domaine par domaine** à l'exécution (`chrome.permissions.request`, déclenchée par un geste de l'utilisateur), car `activeTab` ne suffit pas pour un parcours sur plusieurs pages en mode automatique

Ne sont pas demandés : `cookies`, `webRequest`, `history`, `<all_urls>` en permission obligatoire. La permission `unlimitedStorage` est évaluée en phase 3 et n'est ajoutée que si elle est justifiée et documentée. L'utilisateur peut révoquer l'accès à un domaine depuis les paramètres.

4.4 Arborescence indicative

```
smart-web-extractor/
├── manifest.config.ts            # manifeste généré par le build
├── src/
│   ├── background/
│   │   ├── service-worker.ts
│   │   ├── run-state-machine.ts
│   │   └── message-router.ts
│   ├── content/
│   │   ├── detector.ts
│   │   ├── table-extractor.ts
│   │   ├── list-extractor.ts       # listes non tabulaires
│   │   ├── field-extractor.ts
│   │   ├── locator.ts
│   │   ├── picker-overlay.ts       # sélecteur visuel (Shadow DOM)
│   │   └── action-recorder.ts
│   ├── automation/
│   │   ├── scenario-engine.ts
│   │   ├── navigation.ts
│   │   ├── pagination.ts
│   │   ├── conditions.ts           # conditions de réussite déclaratives
│   │   ├── rate-limiter.ts
│   │   └── retry.ts
│   ├── association/
│   │   ├── keys.ts
│   │   └── merge.ts
│   ├── storage/
│   │   ├── settings.ts
│   │   ├── scenarios.ts
│   │   ├── migrations.ts
│   │   ├── runs.ts
│   │   ├── results.ts
│   │   └── logs.ts
│   ├── export/
│   │   ├── excel-exporter.ts
│   │   ├── csv-exporter.ts
│   │   └── sanitize.ts
│   ├── ui/
```

```
│   │   ├── popup/
│   │   ├── sidepanel/
│   │   ├── dashboard/
│   │   └── components/
│   └── shared/
│       ├── types.ts
│       ├── schemas.ts
│       └── messages.ts
├── tests/
│   ├── unit/
│   └── e2e/
├── test-pages/                    # pages HTML de démonstration servies en HTTP local
├── package.json
├── tsconfig.json
├── vite.config.ts
└── README.md
```

Les pages de test sont servies par un serveur HTTP local (par exemple `vite preview` ou `npx http-server`) et non ouvertes en `file://`. Ce dernier mode exige d'activer manuellement l'accès aux URL de fichiers.

5. Modèle de données

Les scénarios (règles), les exécutions (campagnes) et les résultats (données) sont trois objets distincts.

5.1 Types TypeScript de référence

```
// ---------- Localisateurs ----------
type LocatorStrategy =
  | { kind: 'id'; value: string }
  | { kind: 'attribute'; name: string; value: string }
  | { kind: 'role'; role: string; name?: string }
  | { kind: 'label'; text: string }
  | { kind: 'css'; selector: string }
  | { kind: 'context'; anchorText: string; relation: 'sameRow' | 'next' | 'parent' | 'columnOfHeader' };

interface Locator {
  strategies: LocatorStrategy[];   // ordonnées par fiabilité décroissante
  scope: 'page' | 'row';           // 'row' = relatif à la ligne courante d'une liste
  expectedCount: 'one' | 'many';
}

// ---------- Conditions de réussite (déclaratives, jamais de code) ----------
type SuccessCondition =
  | { kind: 'elementVisible'; locator: Locator }
  | { kind: 'elementGone'; locator: Locator }
  | { kind: 'urlMatches'; pattern: string }            // motif glob, pas de regex arbitraire
  | { kind: 'rowCountChanged' }
  | { kind: 'textChanged'; locator: Locator };

interface RetryPolicy {
  maxAttempts: number;             // 1 à 5
  backoffMs: number;               // délai entre tentatives
  onFailure: 'skipRecord' | 'pause' | 'switchToAssisted' | 'abort';
}

// ---------- Étapes ----------
type StepType =
  | 'locateList' | 'openDetail' | 'extractFields'
  | 'returnToList' | 'nextPage' | 'waitFor' | 'scrollLoad';

interface Step {
  id: string;
  type: StepType;
  locator?: Locator;
```

```
  openIn?: 'sameTab' | 'backgroundTab';  // pour openDetail
  returnMethod?: 'historyBack' | 'locator' | 'closeTab'; // pour returnToList
  successCondition: SuccessCondition;
  timeoutMs: number;                     // 1 000 à 60 000
  retry: RetryPolicy;
}

// ---------- Champs ----------
interface FieldRule {
  key: string;                     // identifiant technique stable, [a-z0-9_]
  label: string;                   // libellé affiché et renommable
  source: 'list' | 'detail' | 'both';
  valueType: 'text' | 'number' | 'date' | 'url';
  required: boolean;
  sensitive: boolean;              // si vrai : jamais lu ni stocké
  listLocator?: Locator;
  detailLocator?: Locator;
  conflictPolicy: 'preferDetail' | 'preferList' | 'flag'; // si source = 'both'
}

// ---------- Association ----------
interface AssociationKey {
  primary:
    | { kind: 'technical'; locator: Locator }
    | { kind: 'field'; fieldKeys: string[] };  // un champ ou une combinaison
  fallback: 'detailUrl' | 'none';
}

// ---------- Pagination ----------
type PaginationRule =
  | { kind: 'none' }
  | { kind: 'nextButton'; locator: Locator }
  | { kind: 'pageLinks'; locator: Locator }
  | { kind: 'infiniteScroll'; maxScrolls: number };

// ---------- Scénario ----------
interface Scenario {
  schemaVersion: number;           // version du format (migrations)
  id: string;
  revision: number;                // révision du scénario, incrémentée à chaque modification validée
  name: string;
  domain: string;                  // ex. "intranet.exemple.local"
  startUrl: string;                // doit appartenir à `domain`
  recordType: 'list' | 'detail' | 'list_and_details';
  compatibleModes: Array<'semi_automatic' | 'automatic'>;
  status: 'draft' | 'validated';
  fields: FieldRule[];
  steps: Step[];
  pagination: PaginationRule;
  association: AssociationKey;
  limits: {
    maxRecords: number;            // nombre d'enregistrements finaux (fusionnés)
    maxPages: number;
    delayBetweenActionsMs: number;  // minimum 300
    maxActionsPerMinute: number;
  };
  duplicates: 'flagOnly' | 'flagAndExcludeFromExport';
  createdAt: string;               // ISO 8601
  updatedAt: string;
}

// ---------- Exécution (campagne) ----------
interface Run {
  id: string;
  scenarioId: string;
  scenarioRevision: number;
  mode: 'semi_automatic' | 'automatic';
  state: 'running' | 'paused' | 'assisted' | 'completed' | 'stopped' | 'failed';
  progress: { page: number; rowIndex: number; recordsDone: number; errors: number };
```

```
  startedAt: string;
  endedAt?: string;
  tabId?: number;
}

// ---------- Résultats ----------
interface FieldValue {
  original: string | null;          // valeur extraite, jamais modifiée
  corrected?: string;               // correction manuelle éventuelle
  listValue?: string | null;        // si source = 'both'
  detailValue?: string | null;
  conflict?: boolean;
}

type RecordStatus =
  | 'complete'        // tous les champs obligatoires présents
  | 'incomplete'      // au moins un champ obligatoire manquant
  | 'error'           // échec d'extraction ou fiche inaccessible
  | 'duplicate'       // même clé qu'un autre enregistrement de l'exécution
  | 'uncertainMatch'; // association liste–fiche non fiable

interface ResultRecord {
  id: string;
  runId: string;
  scenarioId: string;
  scenarioRevision: number;
  recordKey: string;
  keyKind: 'technical' | 'field' | 'detailUrl';
  values: Record<string, FieldValue>;
  sourceUrl: string;                // URL de la fiche, ou de la liste
  listPageUrl?: string;
  collectedAt: string;
  status: RecordStatus[];           // plusieurs états possibles
  validation: 'unreviewed' | 'validated' | 'rejected';
  deleted: boolean;                 // suppression logique
  errors: Array<{ stepId?: string; code: string; message: string }>;
}

// ---------- Journal ----------
interface LogEntry {
  runId: string;
  at: string;
  stepId?: string;
  level: 'info' | 'warning' | 'error';
  code: string;                     // ex. FIELD_NOT_FOUND, AMBIGUOUS_LOCATOR
  message: string;                  // sans aucune valeur extraite sensible
}
```

5.2 Exemple de scénario

```
{
  "schemaVersion": 1,
  "id": "scenario_agents",
  "revision": 3,
  "name": "Extraction des agents",
  "domain": "intranet.exemple.local",
  "startUrl": "https://intranet.exemple.local/agents",
  "recordType": "list_and_details",
  "compatibleModes": ["semi_automatic", "automatic"],
  "status": "validated",
  "fields": [
    { "key": "matricule", "label": "Matricule", "source": "both", "valueType": "text",
      "required": true, "sensitive": false, "conflictPolicy": "flag" },
    { "key": "nom", "label": "Nom", "source": "detail", "valueType": "text",
      "required": true, "sensitive": false, "conflictPolicy": "preferDetail" },
    { "key": "grade", "label": "Grade", "source": "detail", "valueType": "text",
      "required": false, "sensitive": false, "conflictPolicy": "preferDetail" }
```

```
  ],
  "steps": [
    { "id": "s1", "type": "locateList", "successCondition": { "kind": "rowCountChanged" },
      "timeoutMs": 10000, "retry": { "maxAttempts": 2, "backoffMs": 1000, "onFailure": "pause" } },
    { "id": "s2", "type": "openDetail", "openIn": "backgroundTab",
      "successCondition": { "kind": "urlMatches", "pattern": "*/agents/*" },
      "timeoutMs": 15000, "retry": { "maxAttempts": 3, "backoffMs": 2000, "onFailure": "skipRecord" } },
    { "id": "s3", "type": "extractFields", "successCondition": { "kind": "rowCountChanged" },
      "timeoutMs": 10000, "retry": { "maxAttempts": 2, "backoffMs": 1000, "onFailure": "skipRecord" } },
    { "id": "s4", "type": "returnToList", "returnMethod": "closeTab",
      "successCondition": { "kind": "rowCountChanged" },
      "timeoutMs": 10000, "retry": { "maxAttempts": 2, "backoffMs": 1000, "onFailure": "switchToAssisted" } }
  ],
  "pagination": { "kind": "nextButton", "locator": { "strategies": [], "scope": "page", "expectedCount": "one"
} },
  "association": { "primary": { "kind": "field", "fieldKeys": ["matricule"] }, "fallback": "detailUrl" },
  "limits": { "maxRecords": 500, "maxPages": 50, "delayBetweenActionsMs": 500, "maxActionsPerMinute": 60 },
  "duplicates": "flagOnly",
  "createdAt": "2026-10-22T09:00:00Z",
  "updatedAt": "2026-10-22T10:30:00Z"
}
```

Cet exemple illustre la structure. Les localisateurs (volontairement vides ici) et les conditions de réussite réelles sont enregistrés et validés sur le site cible.

5.3 Quotas de stockage

- `chrome.storage.local` est limité (10 Mo par défaut). Il ne contient que les scénarios et les préférences ; la taille d'un scénario est plafonnée (par exemple 256 Ko).
- IndexedDB : en cas de dépassement de quota (`QuotaExceededError`), l'exécution est mise en pause avec un message clair, et la progression déjà enregistrée est conservée. L'espace utilisé est affiché dans les paramètres.

6. Sécurité, confidentialité et conformité

6.1 Sécurité

- Demander uniquement les permissions indispensables (§4.3) et privilégier l'accès aux sites à la demande, domaine par domaine.
- Ne jamais envoyer de données extraites à un serveur externe. Aucune télémétrie.
- Ne jamais collecter ni stocker de mots de passe, cookies, jetons de session ou en-têtes d'authentification.
- Ne jamais lire ni enregistrer de valeurs sensibles (cf. glossaire), ni pendant l'apprentissage, ni pendant l'extraction.
- Ne pas exécuter de code provenant d'une page web, ni utiliser `eval` ou `new Function`. Les conditions de réussite et les motifs d'URL sont déclaratifs.
- Valider tous les messages entre composants et tous les fichiers importés à l'aide de schémas stricts.
- Traiter le contenu des pages comme non fiable : le texte extrait est affiché comme du texte, jamais injecté en HTML (`innerHTML` interdit sur des données extraites).
- Respecter les conditions d'utilisation des sites. Ne jamais contourner un CAPTCHA, une authentification ou une restriction d'accès : leur détection met l'exécution en pause.

6.2 Données personnelles

Les données extraites peuvent être des données personnelles (noms, matricules, grades…). L'extension doit permettre à l'utilisateur de respecter la réglementation qui lui est applicable (RGPD, ou loi nationale de protection des données personnelles).

- **Minimisation** : seuls les champs explicitement sélectionnés sont extraits.

- **Durée de conservation** : durée configurable (30 jours par défaut) au-delà de laquelle les exécutions et résultats sont purgés automatiquement, après notification.
- **Suppression** : l'utilisateur peut supprimer une exécution, tous les résultats d'un scénario ou toutes les données de l'extension, avec confirmation.
- **Journaux** : ils ne contiennent ni valeur extraite ni donnée personnelle, seulement des codes d'étape, des URL sans paramètres et des messages techniques.
- **Chiffrement au repos** : hors périmètre V1. Les données sont protégées par le profil Chrome et la session du système d'exploitation. Cette limite est documentée dans le README.
- Un avertissement est affiché au premier lancement : l'utilisateur est responsable de la licéité de l'extraction et du traitement des données.

7. Gestion des erreurs et robustesse

Problème	Détection	Comportement attendu
Champ introuvable	Aucune stratégie ne trouve l'élément	Valeur manquante (`null`) ; enregistrement marqué `incomplete` si le champ est obligatoire ; journalisé.
Plusieurs correspondances	Plusieurs éléments trouvés pour un localisateur `one`	Semi-automatique : demander à l'utilisateur. Automatique : appliquer la règle validée si elle existe, sinon passer en mode assisté.
Page lente	Condition de réussite non atteinte	Attendre jusqu'au délai maximal, puis appliquer la politique de reprise.
Navigation interrompue	Échec de chargement, onglet rechargé	Mettre en pause et proposer de reprendre à la dernière étape validée.
Fiche inaccessible	Erreur HTTP, page vide	Journaliser, marquer l'enregistrement `error` et passer au suivant si la politique est `skipRecord`. Mise en pause après 3 erreurs consécutives.
Session expirée	Redirection vers une page de connexion, champ mot de passe détecté	Mise en pause immédiate. L'utilisateur se reconnecte lui-même, puis reprend.
CAPTCHA ou contrôle anti-robot	Éléments connus (iframes reCAPTCHA, hCaptcha…), page de défi	Mise en pause immédiate. Aucune tentative de contournement.
Limitation par le serveur (HTTP 429, 403 répétés)	Statut ou contenu de la page	Mise en pause ; proposer d'augmenter le délai entre actions.
Boîte de dialogue `alert`, `confirm`, `beforeunload`	Page bloquée	Mise en pause et demande à l'utilisateur. Pas de réponse automatique.
Onglet fermé ou modifié par l'utilisateur	Événements `chrome.tabs`	Mise en pause ; reprise possible dans un nouvel onglet.
Service worker redémarré, navigateur fermé	État `running` trouvé au démarrage	L'exécution passe à `paused` ; reprise proposée.
Fin de pagination	Bouton absent ou désactivé, ou `maxPages` atteint	Arrêt propre ; exécution `completed`.
Doublon détecté	Clé déjà présente dans l'exécution	Enregistrement marqué `duplicate` (non supprimé).

Problème	Détection	Comportement attendu
Structure du site modifiée	Plus de 50 % des champs obligatoires introuvables sur 3 enregistrements consécutifs	Arrêt de l'exécution ; scénario marqué « à vérifier ».
Quota de stockage dépassé	`QuotaExceededError`	Mise en pause, message clair, état préservé.

Le journal local enregistre l'étape, l'heure, le niveau, le code et le message, sans donnée sensible (§6.2).

7.1 Limites de sécurité

- Limite configurable du nombre d'enregistrements (`maxRecords`) et de pages (`maxPages`).
- Tentatives limitées (5 au maximum par étape) et fréquence d'action bornée (`delayBetweenActionsMs` ≥ 300 ms, `maxActionsPerMinute`).
- Pause, reprise et **arrêt immédiat** accessibles à tout moment depuis le panneau latéral et par raccourci clavier.
- Aucune boucle de clics sans condition de réussite vérifiée. Un même élément ne peut pas être cliqué plus de N fois de suite sans que la page change.
- Passage en mode assisté dès qu'une action ne peut pas être automatisée de façon fiable.

7.2 Reprise sans doublons

- La progression est persistée après chaque étape (§4.2).
- À la reprise, les enregistrements déjà présents (même `recordKey` dans la même exécution) sont ignorés, et non recréés.
- La relance des enregistrements en erreur met à jour l'enregistrement existant au lieu d'en créer un nouveau.

8. Tests et critères d'acceptation

8.1 Tests obligatoires

- Détection des tableaux HTML (avec et sans en-têtes, cellules fusionnées) et de leurs colonnes.
- Détection des listes non tabulaires.
- Détection des champs et de leurs libellés ; exclusion des valeurs sensibles.
- Extraction avec des valeurs différentes de celles de la page d'apprentissage.
- Association liste–fiche par clé unique, par combinaison de champs et par clé de repli (avec avertissement).
- Conflit de valeurs entre liste et fiche.
- Sauvegarde, rechargement, révision, import et export d'un scénario ; rejet d'un import invalide.
- Migration d'un scénario d'une version de schéma antérieure.
- Rejeu sur plusieurs fiches de test.
- Pagination, défilement infini et contenu dynamique.
- Champs manquants, doublons et localisateurs ambigus.
- Export XLSX et CSV avec accents, caractères arabes, identifiants à zéros initiaux et valeurs commençant par `=`, `+`, `-`, `@`.
- Reprise après interruption (y compris redémarrage du service worker) sans créer de doublons.
- Mise en pause lors d'une session expirée et d'un CAPTCHA simulés.
- Validation des messages entre composants (messages malformés rejetés).
- Vérification des permissions déclarées dans le manifeste généré.
- Purge des données et respect de la durée de conservation.

8.2 Pages de test locales

Servies en HTTP local depuis `test-pages/` :

- liste de 25 lignes au moins ;
- liste sans `<thead>` et liste avec cellules fusionnées ;
- liste en cartes (non tabulaire) ;
- fiches détaillées ouvertes par lien (`href`) et par clic JavaScript (sans `href`) ;
- pagination sur 3 pages au moins ;
- contenu chargé avec un délai aléatoire, et liste à défilement infini ;
- fiches avec champs manquants ;
- page avec localisateurs ambigus (plusieurs éléments identiques) ;
- formulaire contenant un champ mot de passe (pour vérifier qu'il est exclu) ;
- page de connexion simulée (session expirée) et page de défi simulée (CAPTCHA factice).

8.3 Critères d'acceptation

#	Critère	Mesure
1	Build et chargement	`npm run build` sans erreur ni avertissement TypeScript ; extension chargée dans Chrome sans erreur dans `chrome://extensions`.
2	Détection	100 % des tableaux et listes des pages de test détectés ; 100 % des colonnes correctement nommées.
3	Sélection	Les champs peuvent être sélectionnés, renommés, marqués obligatoires ou sensibles.
4	Scénarios	Création, révision, rechargement après redémarrage du navigateur, import et export vérifiés.
5	Modes	Les deux modes extraient 100 % des enregistrements attendus sur le parcours de démonstration (3 pages, 25 fiches au moins).
6	Association	0 association erronée sur les jeux de test ; associations incertaines toutes signalées.
7	Erreurs	Chaque cas du §7 est couvert par un test. Aucun cas ne provoque de navigation hors du domaine du scénario ni de clic non vérifié.
8	Reprise	Après une interruption forcée à n'importe quelle étape, la reprise produit exactement le même jeu de résultats, sans doublon.
9	Export	Les fichiers XLSX et CSV rouverts par les tests conservent accents, arabe et zéros initiaux ; aucune valeur exportée n'est interprétée comme une formule.
10	Confidentialité	Aucune requête réseau émise par l'extension (vérifié en test de bout en bout) ; suppression totale des données vérifiée.
11	Accessibilité	Interfaces utilisables entièrement au clavier ; conformité WCAG 2.1 niveau AA vérifiée sur les écrans principaux (contrastes, libellés, focus).
12	Interface	Interface intégralement en français ; panneau latéral utilisable de 320 px à 600 px de large.
13	Documentation	README : installation, build, chargement, tests, permissions, limites connues.

9. Plan de développement

Chaque phase s'achève par une démonstration et une validation avant de passer à la suivante. Les durées sont à estimer par l'équipe.

Phase	Travaux	Livrable	Critère de sortie
1. Fondations	Projet Vite + TypeScript strict, plugin d'extension, manifeste MV3, permissions, panneau latéral, popup, routeur de messages validés	Extension minimale installable	Critère 1 ; message aller-retour panneau ↔ service worker ↔ content script vérifié
2. Détection	Tableaux, listes, champs, sélecteur visuel, localisateurs, aperçu, ambiguïtés	Extraction manuelle d'une page testée	Critères 2 et 3 sur les pages de test
3. Stockage	Scénarios, révisions, migrations, exécutions, résultats, quotas, purge	Scénarios et résultats persistants	Critère 4 ; tests de quota et de purge
4. Apprentissage	Enregistrement des étapes, conditions de réussite, éditeur de scénario, mode essai	Scénario modifiable et testable	Scénario de démonstration enregistré puis rejoué en mode essai
5. Automatisation	Machine à états, navigation, pagination, attentes, reprise, mode assisté, limites	Extraction automatisée sur les pages de test	Critères 5, 7 et 8
6. Résultats et export	Association, conflits, tableau de résultats, corrections, exports XLSX et CSV	Exports opérationnels	Critères 6 et 9
7. Finalisation	Tests de bout en bout complets, revue de sécurité, accessibilité, documentation	Version utilisable et documentée	Tous les critères du §8.3

10. Prompt initial pour Claude Code

À copier dans Claude Code après avoir ouvert le dossier du projet, avec ce cahier des charges placé à la racine sous le nom `CAHIER_DES_CHARGES.md`.

```
# Projet : Smart Web Extractor — Extension Chrome

Tu es un développeur senior spécialisé en extensions Chrome Manifest V3, TypeScript,
React, automatisation de navigateur, extraction HTML, sécurité et tests.

Le cahier des charges complet se trouve dans CAHIER_DES_CHARGES.md. Lis-le entièrement
avant de commencer : il fait référence pour toutes les règles (modèle de données §5,
contraintes MV3 §4.2, permissions §4.3, export §3.7, erreurs §7, critères §8.3).

## Objectif
Extension permettant d'extraire des données de sites auxquels l'utilisateur a légitimement
accès, sans script spécifique par site : analyse de la page, détection des tableaux, listes
et champs, sélection, enregistrement explicite d'un parcours, validation d'un scénario
versionné, puis exécution semi-automatique ou automatique (mode choisi au lancement).
Association liste–fiche par clé stable, résultats consultables et corrigeables, export XLSX/CSV.

## Points d'attention impératifs
- Pilotage dans un panneau latéral (chrome.sidePanel) ; la popup sert seulement au lancement.
- Service worker éphémère : aucun état uniquement en mémoire ; état persisté après chaque étape.
- Content scripts sans état, injectés à la demande via chrome.scripting, compilés en IIFE.
- IndexedDB uniquement dans le contexte de l'extension (jamais depuis un content script).
- Messages et imports validés par schéma ; contenu des pages traité comme non fiable.
- Conditions de réussite déclaratives ; pas d'eval ni de new Function ; pas d'innerHTML sur des données
extraites.
- Permissions minimales ; accès aux sites demandé domaine par domaine (optional_host_permissions).
- Aucune donnée envoyée à l'extérieur ; jamais de mot de passe, cookie, jeton ni valeur sensible.
- Doublons signalés et non supprimés ; valeur d'origine conservée lors des corrections.
- CSV : UTF-8 avec BOM, séparateur ';' par défaut, protection contre les formules par apostrophe,
  jamais d'astuce ="...". XLSX : valeurs en texte, filtres, première ligne figée, option RTL.
- Mise en pause sur session expirée, CAPTCHA, HTTP 429/403, boîte de dialogue ; aucun contournement.
```

```
- Limites : maxRecords, maxPages, délai minimal entre actions, tentatives bornées, pause et arrêt immédiat.
- Interface en français, accessible au clavier, WCAG 2.1 AA.

## Tests
Crée les pages de démonstration du §8.2 dans test-pages/ (servies en HTTP local).
Vitest pour les tests unitaires, Playwright (Chromium, contexte persistant, extension chargée)
pour les tests de bout en bout. Couvre la liste du §8.1.

## Méthode de travail
Ne développe pas tout d'un seul coup.
1. Examine l'environnement (versions de Node, npm, Chrome) et vérifie les versions compatibles
   des dépendances.
2. Propose l'architecture, les types (à partir du §5.1) et le modèle de données.
3. Réalise les phases du §9 une par une.
4. Après chaque phase : compile, exécute les tests, corrige, documente le test manuel,
   et vérifie le critère de sortie de la phase.
5. Ne prétends jamais qu'une fonctionnalité fonctionne sans l'avoir vérifiée.

Commence par la phase 1 : architecture, arborescence, initialisation du projet,
extension minimale installable avec panneau latéral et routeur de messages, et
vérification du build. Attends une validation avant de poursuivre.
```

11. Installation et démarrage

1. Installer Node.js (version LTS en cours) et npm.
2. Créer le dossier du projet, y placer `CAHIER_DES_CHARGES.md` et l'ouvrir dans VS Code avec Claude Code.
3. Coller le prompt du §10 et valider la phase 1.
4. Installer les dépendances (`npm install`) et lancer le build (`npm run build`).
5. Ouvrir `chrome://extensions`, activer le mode développeur et charger le dossier de build (par exemple `dist/`).
6. Lancer le serveur des pages de test et tester d'abord sur ces pages, avant tout site réel.
7. Sur un site réel : accorder l'accès au domaine lorsque l'extension le demande, et vérifier que l'extraction est autorisée par les conditions d'utilisation du site.

Fin du cahier des charges — Smart Web Extractor v2.0